Aplicaciones con Enfoque Orientado a Servicios

Tema Nº3:
Servicios web REST

Indicador de logro Nº3:
Construye servicios web utilizando el estilo de arquitectura REST

idat

TEMA Nº3:

Servicios web REST

Subtema 3.1:

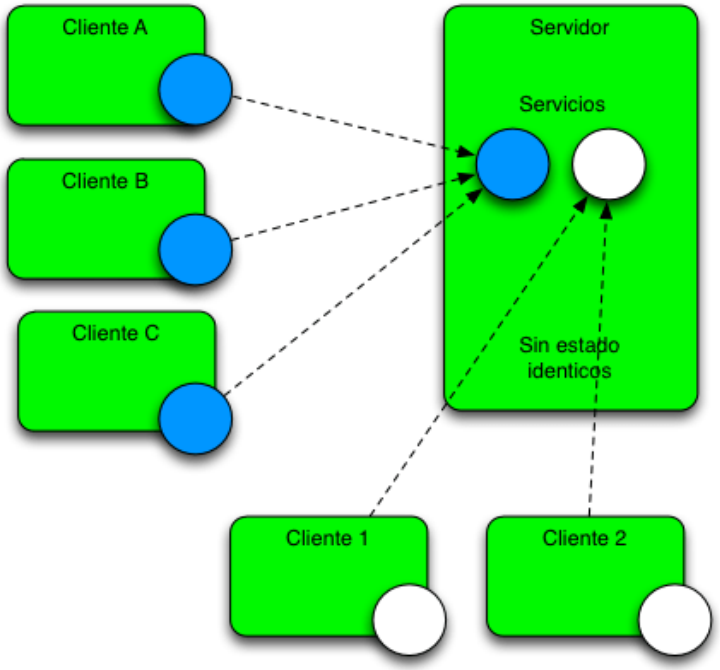
Conceptos básicos y buenas prácticas

El objetivo principal de cualquier sistema distribuido es facilitar el acceso a recursos remotos. *Representational State Transfer* o REST es un estilo de arquitectura utilizado en el desarrollo de servicios web, muy diferente a aquellos que se realizan con el protocolo SOAP. REST se diseñó pensando en ser simple para lograr una rápida adopción del usuario y un desarrollo rápido.

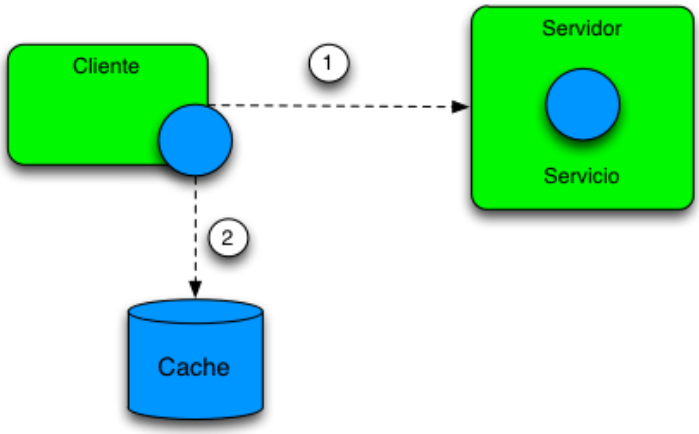
Los servicios que implementan REST tienen las siguientes características:

- Cliente / Servidor
  Se define una interface de comunicación entre ambos separando completamente las responsabilidades de cada parte.

- Sin estado
  Los servicios web REST no mantienen ningún estado asociado al cliente. Cada petición que se realiza es completamente independiente de la siguiente.



- Caché
  El contenido se puede almacenar de tal forma que, una vez realizada la primera petición al servicio, las siguientes peticiones pueden apoyarse en la caché si fuera necesario.



Los servicios web basados en la arquitectura REST, también llamados servicios web RESTful, se basan en **recursos**. Un recurso es una entidad, la cual se almacena principalmente en un servidor y el cliente solicita el recurso utilizando servicios Web RESTful.

A continuación se sugieren buenas prácticas a considerar al momento de desarrollar servicios REST:

- Al devolver una respuesta desde el servidor, agregar el código de estado HTTP correspondiente. Estas respuestas se agrupan en cinco clases:
  - Respuestas informativas (100–199)
  - Respuestas satisfactorias (200–299)
  - Redirecciones (300–399)
  - Errores de los clientes (400–499)
  - errores de los servidores (500–599)

- Utilizar los métodos HTTP según la operación a realizar, no únicamente GET o POST.

- Manejar control de versiones para los servicios para permitir compatibilidad con los clientes.

- Incluir un token de sesión en encabezados en el caso de solicitudes privadas.

- En el caso de resultados extensos, use paginación.

- Definir el servicio para devolver lo que se está solicitando.

- Devolver los tipos de datos correctos, no solo cadenas.

Ejemplos:

2xx success

200
standard response for successful HTTP requests

201
request has been fulfilled; new resource created

202
request accepted, processing pending

203
request processed, information may be from another source

204
request processed, no content returned

205
request processed, no content returned, reset document view

206
partial resource return due to request header

207
XML, can contain multiple separate responses

208
results previously returned

226
request fulfilled, reponse is instance-manipulations

3xx redirection

300
multiple options for the resource delivered

301
this and all future requests directed to the given URI

302
temporary response to request found via alternative URI

303
permanent response to request found via alternative URI

304
resource has not been modified since last requested

305
content located elsewhere, retrieve from there

306
subsequent requests should use the specified proxy

307
connect again to different URI as provided

308
resumable HTTP requests

4xx client error

400
request cannot be fulfilled due to bad syntax

401
authentication is possible but has failed

402
payment required, reserved for future use

Códigos de estado HTTP

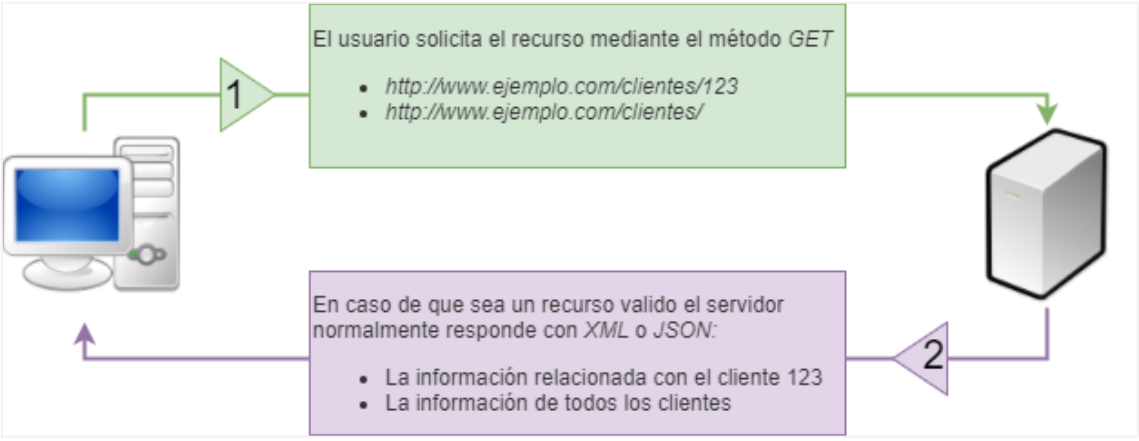
Subtema 3.2:
Operaciones básicas

Para acceder o manipular los recursos del servidor, los servicios web que implementan REST deben utilizar los métodos disponibles en HTTP. Los principales según la operación a realizar son:

Operación	Método HTTP	URI	Parámetros	Resultado
Listar	GET	/{recurso}	-	Lista del tipo del recurso
Crear	POST	/{recurso}	Dentro del cuerpo en el POST	Se crea un nuevo recurso
Leer	GET	/{recurso}/{id}	-	Un recurso en función del id
Actualizar	PUT	/{recurso}/{id}	Dentro del cuerpo en el PUT	Se actualiza el recurso
Borrar	DELETE	/{recurso}/{id}	-	Se elimina el recurso en función del id

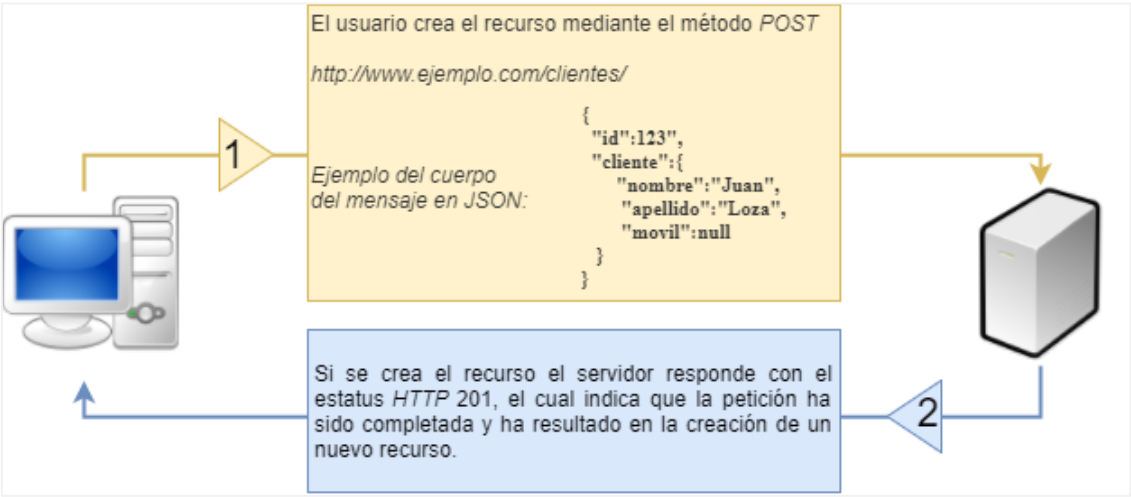
Ejemplos:

GET



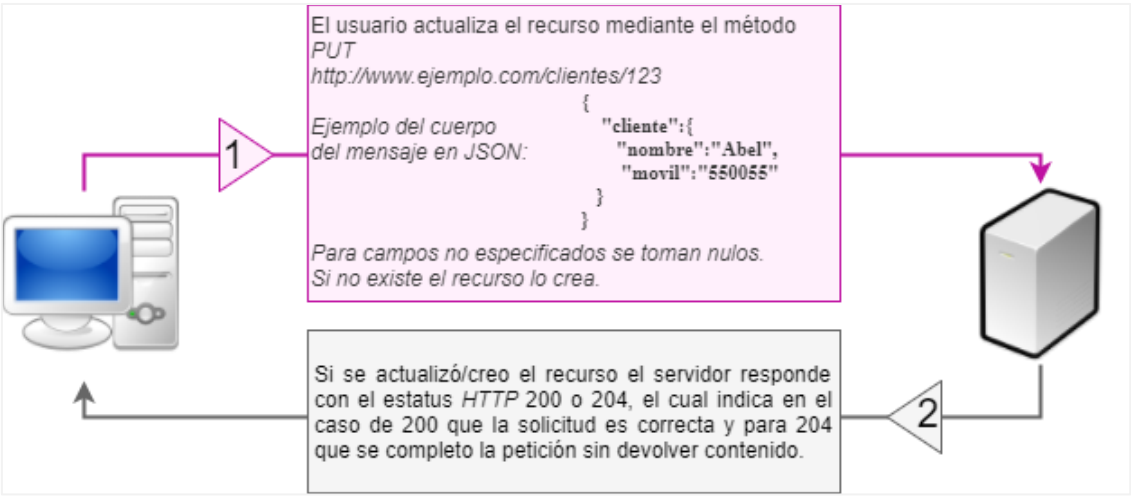
Fuente: https://blog.bi-geek.com

POST



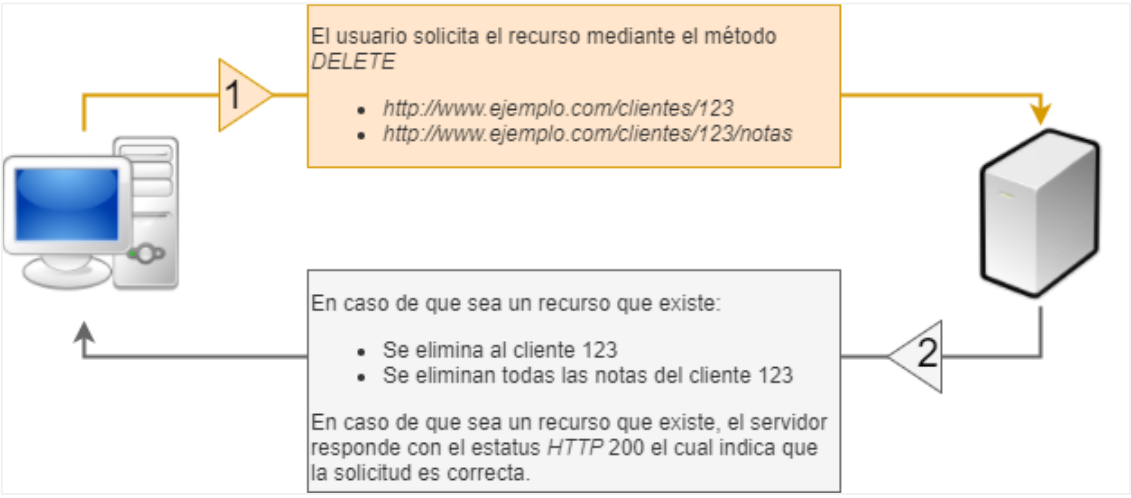
Fuente: https://blog.bi-geek.com

PUT



Fuente: https://blog.bi-geek.com

DELETE



Fuente: https://blog.bi-geek.com

Subtema 3.3:
Formatos básicos de intercambio de información

El formato recomendado para un servicio web RESTful es JSON, sin embargo, también puede retornar la respuesta en formato XML, texto plano, HTML o en secuencia de bytes.

JSON (JavaScript Object Notation) es un formato ligero de intercambio de datos. JSON es de fácil lectura y escritura para los usuarios. Por estas características, es el formato predilecto en REST.

Ejemplos:

El contenido de JSON va entre llaves, y para arreglos o lista de objetos dentro de corchetes. Además, JSON se caracteriza porque se basa en una estructura clave o identificador – valor, sin necesidad de especificar el tipo de dato que contiene el valor.

```
{
  "string": "Hi",
  "number": 2.5,
  "boolean": true,
  "null": null,
  "object": { "name": "Kyle", "age": 24 },
  "array": ["Hello", 5, false, null, { "key": "value", "number": 6 }],
  "arrayOfObjects": [
    { "name": "Jerry", "age": 28 },
    { "name": "Sally", "age": 26 }
  ]
}
```

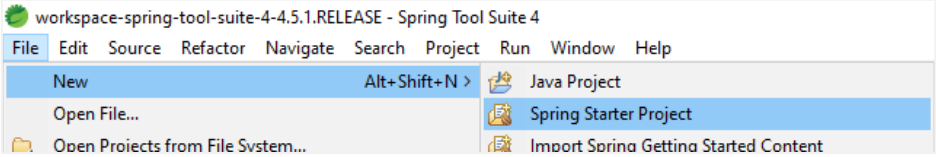
Subtema 3.4:
Creación y consumo de servicios web REST

Para crear un servicio web del tipo RESTful utilizaremos la herramienta Spring Tool Suite, Maven y Java 8.

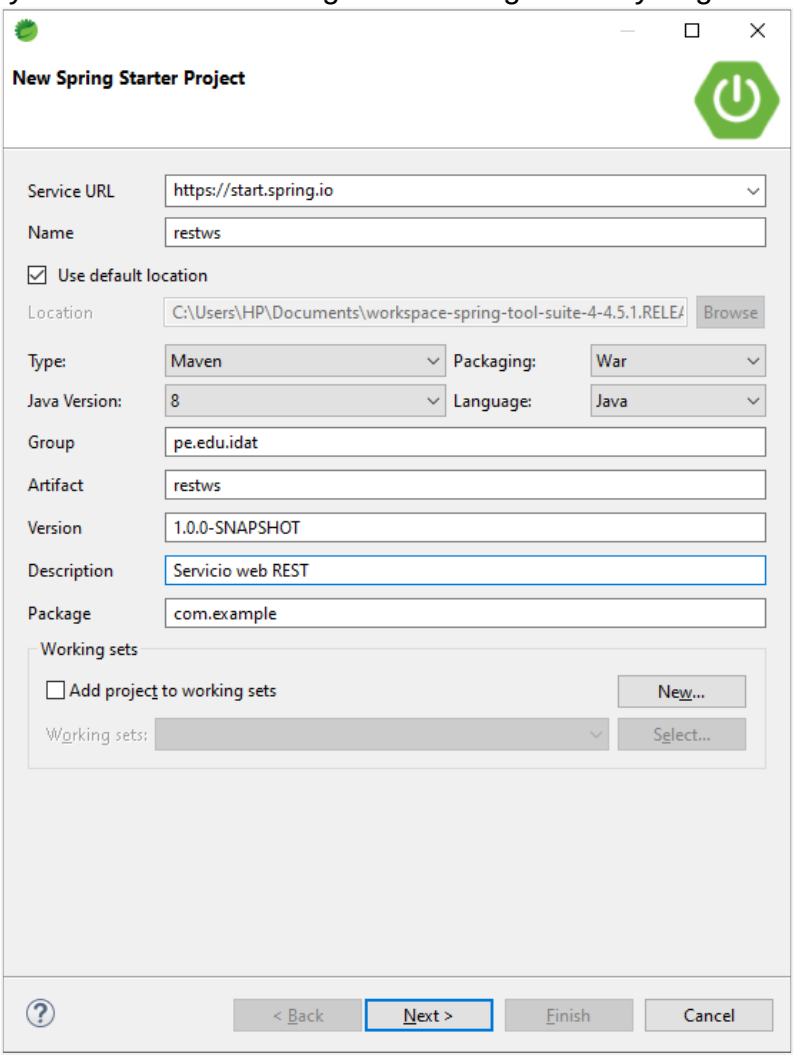
Ejemplos:

Creación de servicio web SOAP

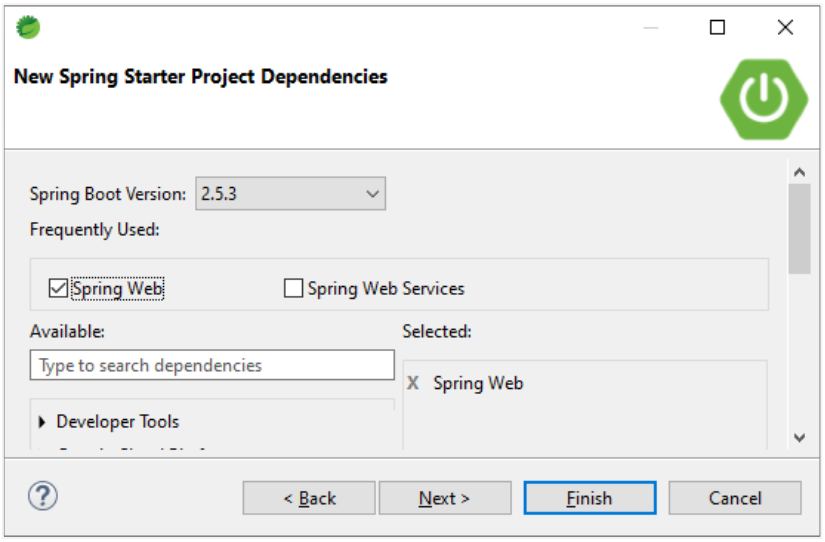
1. En Spring Tool Suite vaya al menú *File* > *New* y seleccione la opción *Spring Starter Project*.



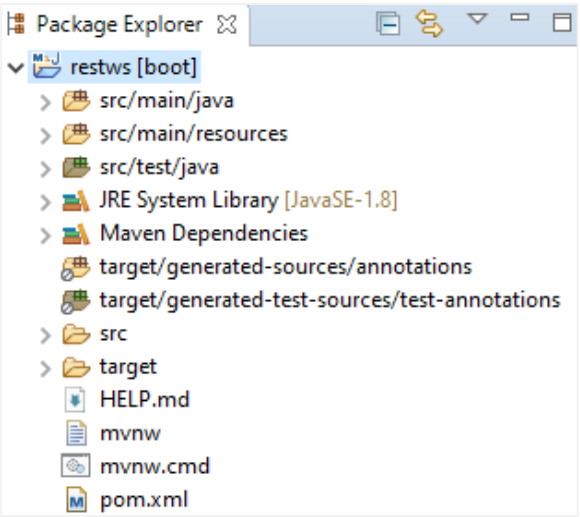
2. Cree el proyecto **restws** con la siguiente configuración y haga clic en Siguiente:



3. Agregue la dependencia Spring Web y haga clic en Siguiente.



4. En la opción de Site Info, haga clic en Finalizar y verifique su proyecto.



Ahora que ha configurado el proyecto y el sistema de compilación, puede crear su servicio web.

El servicio a desarrollar manejará las solicitudes con todos los métodos HTTP para realizar un CRUD básico. Además, deberá devolver el código de estado 200 para peticiones correctas.

5. Añada la clase java **Pais** para modelar la representación de un país. Esta clase debe contener campos, constructores y descriptores de acceso.

Pais.java

```java
package com.example;

public class Pais {

    private long id;
    private String nombre;
    private int poblacion;

    public Pais(long id, String nombre, int poblacion) {
        this.id = id;
        this.nombre = nombre;
        this.poblacion = poblacion;
    }

    public long getId() {
        return id;
    }

    public void setId(long id) {
        this.id = id;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public int getPoblacion() {
        return poblacion;
    }

    public void setPoblacion(int poblacion) {
        this.poblacion = poblacion;
    }
```

6. En el enfoque de Spring para construir servicios web RESTful, las solicitudes HTTP son manejadas por un controlador. Agregue la clase **PaisController** y añada la anotación @RestController y mapearemos a este recurso con el nombre de **países**. Cada método del CRUD hará uso de los verbos HTTP como GET, POST, PUT y DELETE. En esta clase simularemos una carga inicial de 3 países.

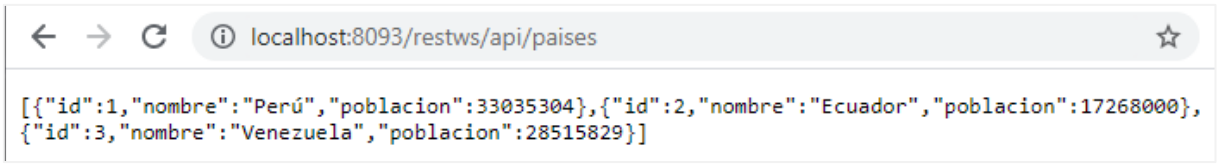
PaisController.java

```java
package com.example;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicLong;

import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
```

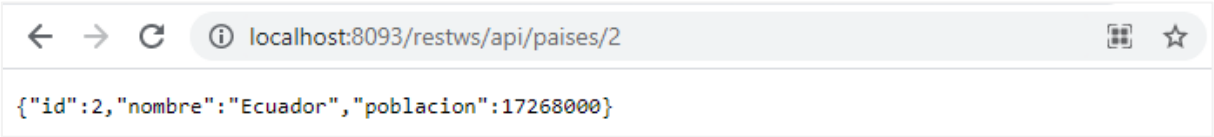
```java
16 import org.springframework.web.bind.annotation.RequestMapping;
17 import org.springframework.web.bind.annotation.RestController;
18
19 @RestController
20 @RequestMapping("api/paises")
21 public class PaisController {
22
23     private static final List<Pais> paises = new ArrayList<>();
24     private final AtomicLong contador = new AtomicLong();
25
26     public PaisController() {
27         initData();
28     }
29
30     // Repositorio de prueba
31     public void initData() {
32         Pais peru = new Pais(contador.incrementAndGet(), "Perú", 33035304);
33         paises.add(peru);
34
35         Pais ecuador = new Pais(contador.incrementAndGet(), "Ecuador", 17268000);
36         paises.add( ecuador);
37
38         Pais venezuela = new Pais(contador.incrementAndGet(), "Venezuela", 28515829);
39         paises.add( venezuela);
40     }
41
42     // Métodos HTTP
43
44     @GetMapping()
45     public ResponseEntity<List<Pais>> listar() {
46         return new ResponseEntity<>(paises, HttpStatus.OK);
47     }
48
49     @GetMapping({"/{id}"})
50     public ResponseEntity<Pais> obtener(@PathVariable long id) {
51         Pais p = paises.stream().filter(x -> id == x.getId()).findAny().orElse(null);
52         if(p != null)
53             return new ResponseEntity<Pais>(p, HttpStatus.OK);
54         else
55             return new ResponseEntity<Pais>(p, HttpStatus.NOT_FOUND);
56     }
57
58     @PostMapping()
59     public ResponseEntity<Pais> registrar(@RequestBody Pais p) {
60         Pais pais = new Pais(contador.incrementAndGet(), p.getNombre(), p.getPoblacion());
61         paises.add(pais);
62
63         HttpHeaders httpHeaders = new HttpHeaders();
64         httpHeaders.add("paises", "api/paises/" +  pais.getId());
65         return new ResponseEntity<>(pais, httpHeaders, HttpStatus.CREATED);
66     }
67
68     @PutMapping({"/{id}"})
69     public ResponseEntity<Pais> actualizar(@PathVariable long id,  @RequestBody Pais p) {
70         Pais temp = null;
71         for (Pais pais : paises) {
72             if(pais.getId() == id) {
73                 pais.setNombre(p.getNombre());
74                 pais.setPoblacion(p.getPoblacion());
75                 temp = pais;
76                 break;
77             }
78         }
79
80         return new ResponseEntity<>(temp, HttpStatus.OK);
81     }
82
83     @DeleteMapping({"/{id}"})
84     public ResponseEntity<Pais> eliminar(@PathVariable long id) {
85         Pais p = paises.stream().filter(x -> id == x.getId()).findAny().orElse(null);
86         if(p != null)
87             paises.remove(p);
88
89         return new ResponseEntity<Pais>(HttpStatus.NO_CONTENT);
90     }
91 }
```

7. Ejecute la aplicación y en su navegador realice las siguientes peticiones:

   - http://localhost:8093/restws/api/paises

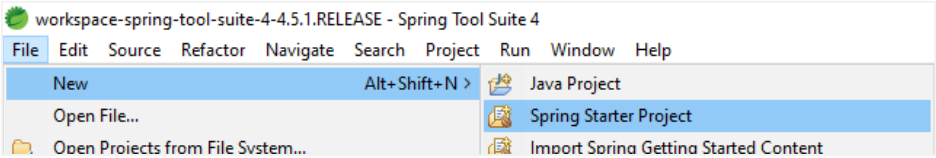


   - http://localhost:8093/restws/api/paises/2

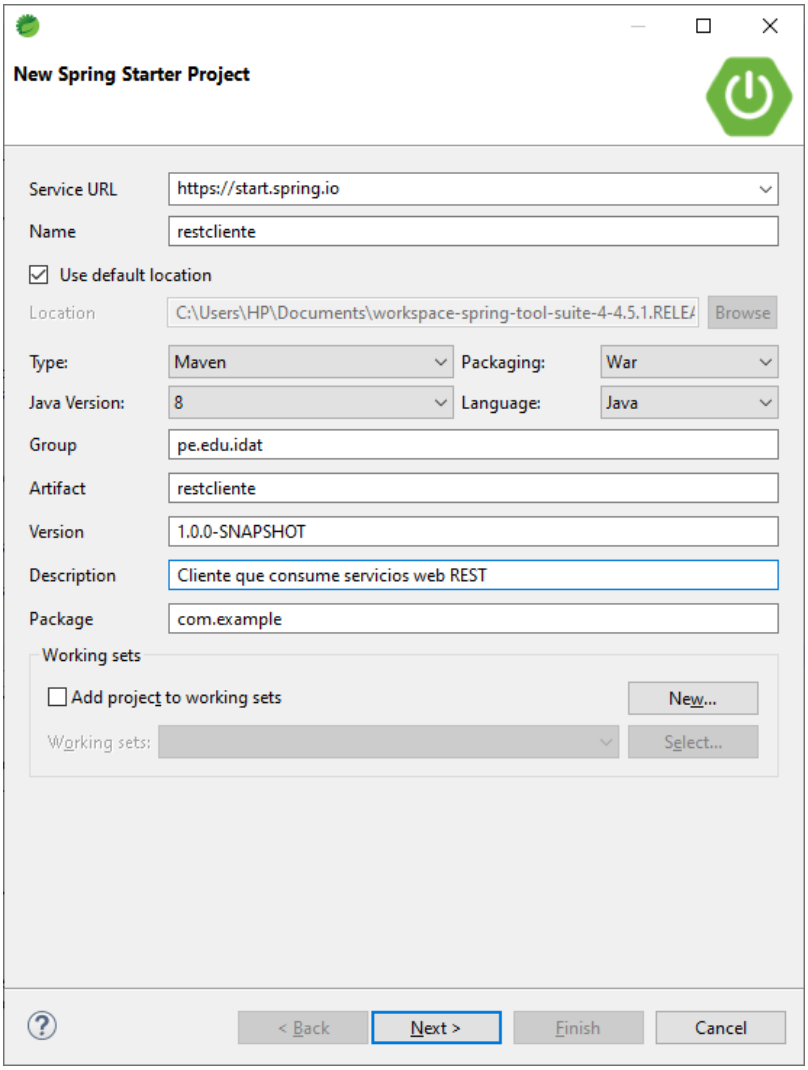


Consumo de servicio web REST

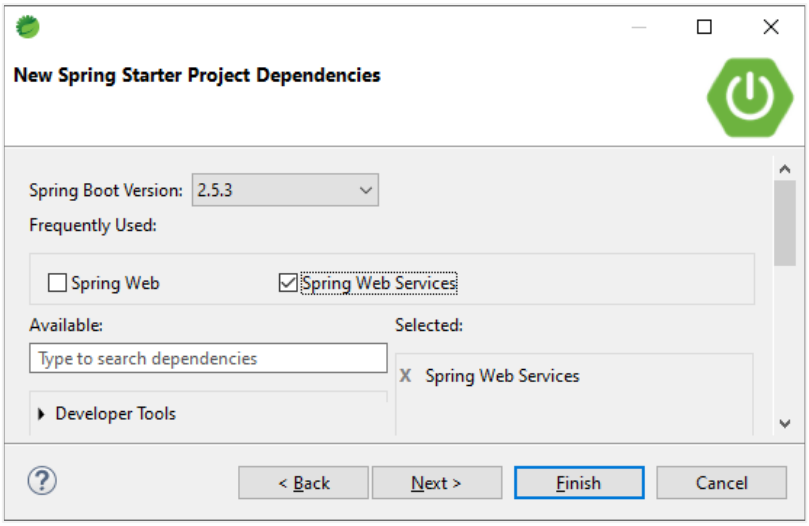
1. En Spring Tool Suite vaya al menú *File* > *New* y seleccione la opción *Spring Starter Project*.



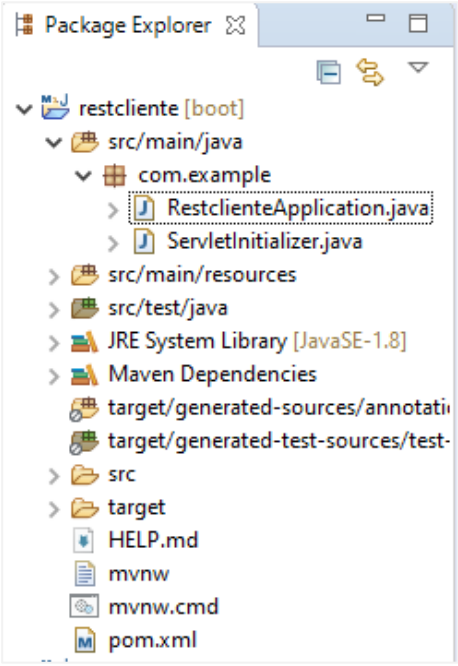
2. Cree el proyecto **restcliente** con la siguiente configuración y haga clic en Siguiente:



3. Agregue la dependencia Spring Web Services y haga clic en Siguiente.



4. En la opción de Site Info, haga clic en Finalizar y verifique su proyecto.



5. Agregue la clase **RestCliente** para realizar el consumo de los servicios web y defina las URL para invocar a los métodos. Agregue el método *callGetAllPaisesAPI* para listar todos los países.

RestCliente.java

```java
package com.example;

import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;

import org.springframework.http.HttpEntity;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpMethod;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.client.RestTemplate;

public class RestCliente {

    private static final String GET_ALL_PAISES_API ="http://localhost:8093/restws/api/paises";
    private static final String GET_PAIS_BY_ID_API ="http://localhost:8093/restws/api/paises/{id}";

    static RestTemplate restTemplate = new RestTemplate();

    public static void main(String[] args) {
        callGetAllPaisesAPI();
    }

    private static void callGetAllPaisesAPI() {
        HttpHeaders headers = new HttpHeaders();
        headers.setAccept(Arrays.asList(MediaType.APPLICATION_JSON));

        HttpEntity<String> entity = new HttpEntity<>("parameters", headers);

        ResponseEntity<String> result = restTemplate.exchange(GET_ALL_PAISES_API, HttpMethod.GET, entity, String.class);
        System.out.println(result);
    }
```

6. Ejecute la clase como JavaApplication y verifique el resultado en la consola.

```
Problems  @ Javadoc  Declaration  Console  Progress  Servers
<terminated> RestCliente [Java Application] C:\Program Files\Java\jre1.8.0_301\bin\javaw.exe (Jul 31, 2021, 2:14:34 AM)
02:14:35.927 [main] DEBUG org.springframework.web.client.RestTemplate - HTTP GET http://localhost:8093/restws/api/paises
02:14:35.949 [main] DEBUG org.springframework.web.client.RestTemplate - Accept=[text/plain, application/json, application/*+json, */*]
02:14:35.953 [main] DEBUG org.springframework.web.client.RestTemplate - Writing [parameters] with org.springframework.http.converter.StringHttpMessageConverter
02:14:36.082 [main] DEBUG org.springframework.web.client.RestTemplate - Response 200 OK
02:14:36.083 [main] DEBUG org.springframework.web.client.RestTemplate - Reading to [java.lang.String] as "application/json"
<200,[{"id":1,"nombre":"Perú","poblacion":33035304},{"id":3,"nombre":"Venezuela","poblacion":28515829},{"id":4,"nombre":"Argentina","poblacion":44939000}],[Content-Type:"application/json", Tr
```

7. Agregue el método *callGetPaisByIdAPI* y añádalo en el método *main*.

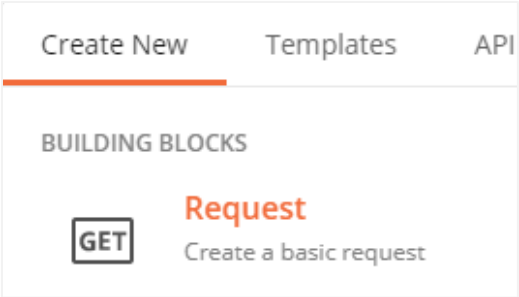
```java
    public static void main(String[] args) {
        callGetAllPaisesAPI();
        System.out.println("\n");
        callGetPaisByIdAPI();
    }

    private static void callGetAllPaisesAPI() {

    private static void callGetPaisByIdAPI() {
        Map<String, Integer> param = new HashMap<>();
        param.put("id", 1);

        String resultado = restTemplate.getForObject(GET_PAIS_BY_ID_API, String.class, param);
        System.out.println(resultado);
    }
```

8. Verifique el resultado en la consola.

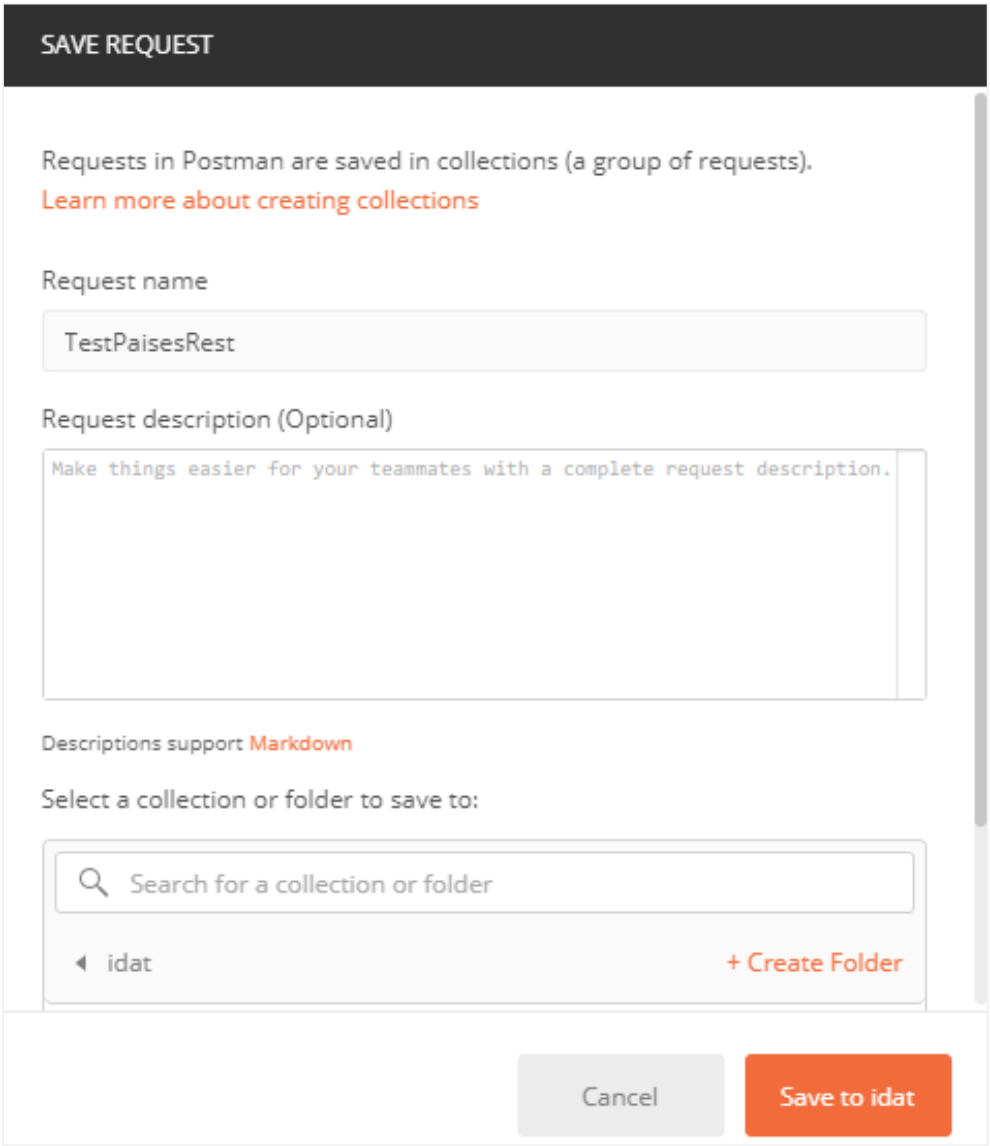
```
Problems  @ Javadoc  Declaration  Console  Progress  Servers
<terminated> RestCliente [Java Application] C:\Program Files\Java\jre1.8.0_301\bin\javaw.exe (Jul 31, 2021, 2:19:55 AM)

02:19:56.455 [main] DEBUG org.springframework.web.client.RestTemplate - HTTP GET http://localhost:8093/restws/api/paises/1
02:19:56.456 [main] DEBUG org.springframework.web.client.RestTemplate - Accept=[text/plain, application/json, application/*+json, */*]
02:19:56.467 [main] DEBUG org.springframework.web.client.RestTemplate - Response 200 OK
02:19:56.467 [main] DEBUG org.springframework.web.client.RestTemplate - Reading to [java.lang.String] as "application/json"
{"id":1,"nombre":"Perú","poblacion":33035304}
```

Pruebas en Postman

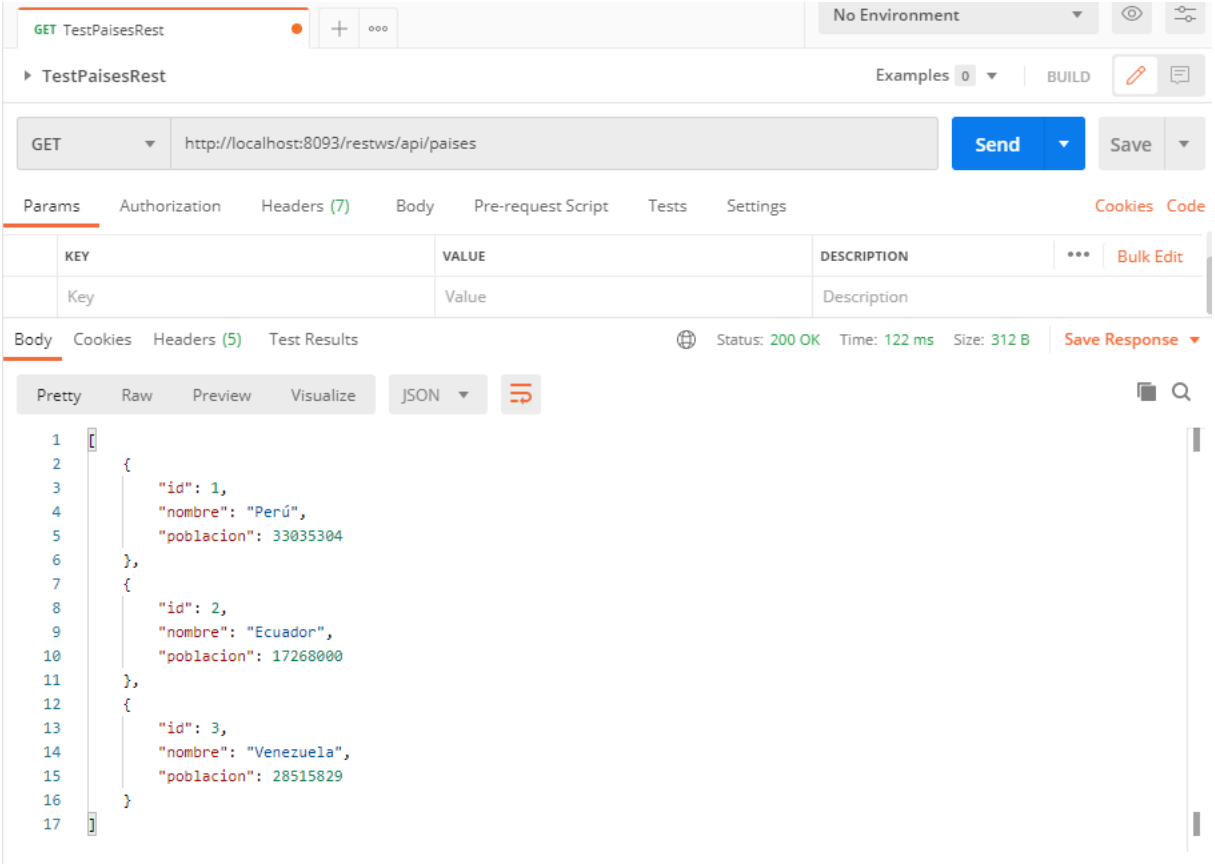
1. En Postman, haga clic en el botón *New* y seleccione la opción *Request*.



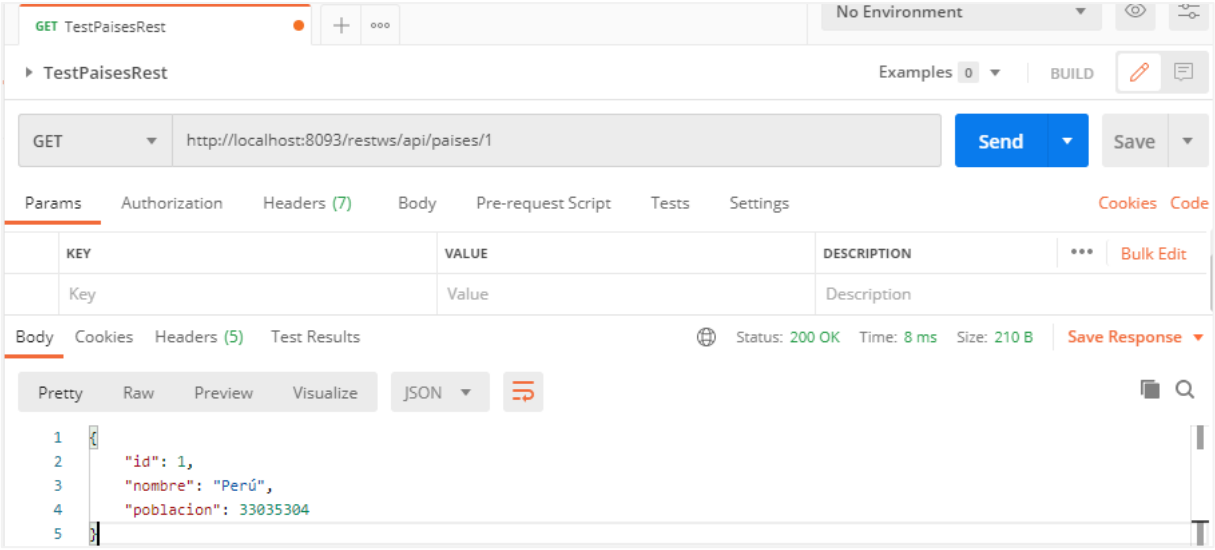
2. En la ventana *Save Request*, ingrese el nombre **TestPaisesRest** y presione OK.



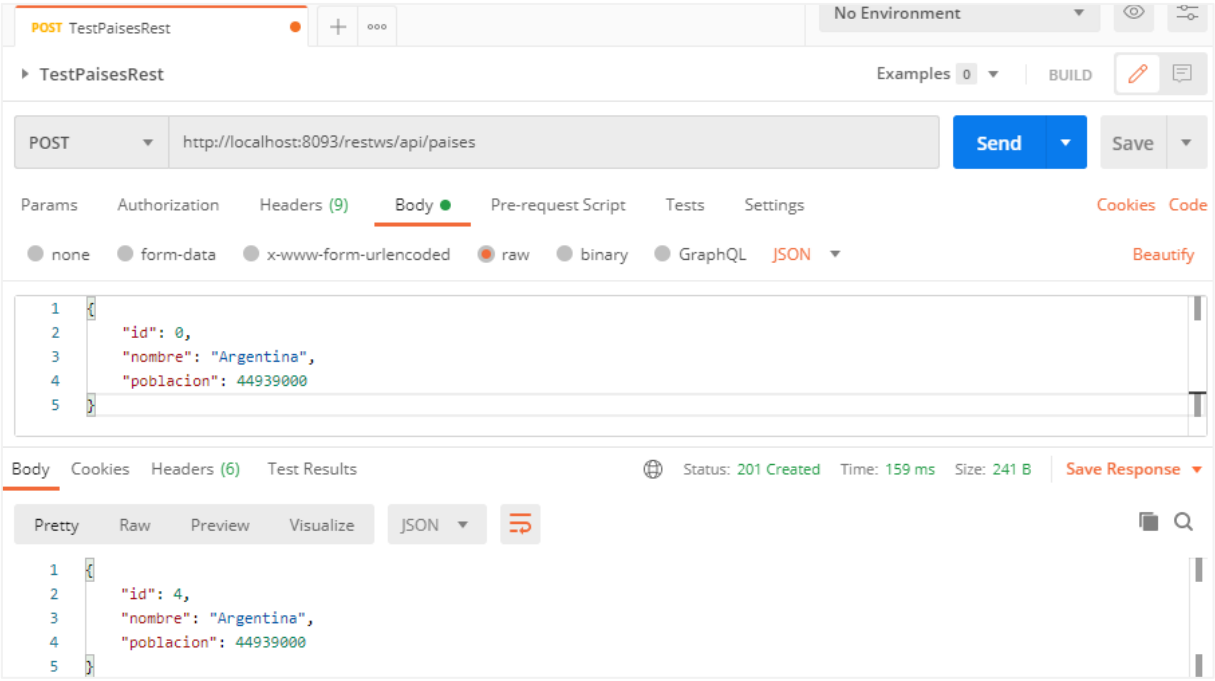
3. Para la petición GET, ingrese la URL http://localhost:8093/restws/api/paises y presione en el botón *Send*.



4. Para obtener un solo país, agregue el parámetro id en la misma petición GET y pruebe nuevamente.

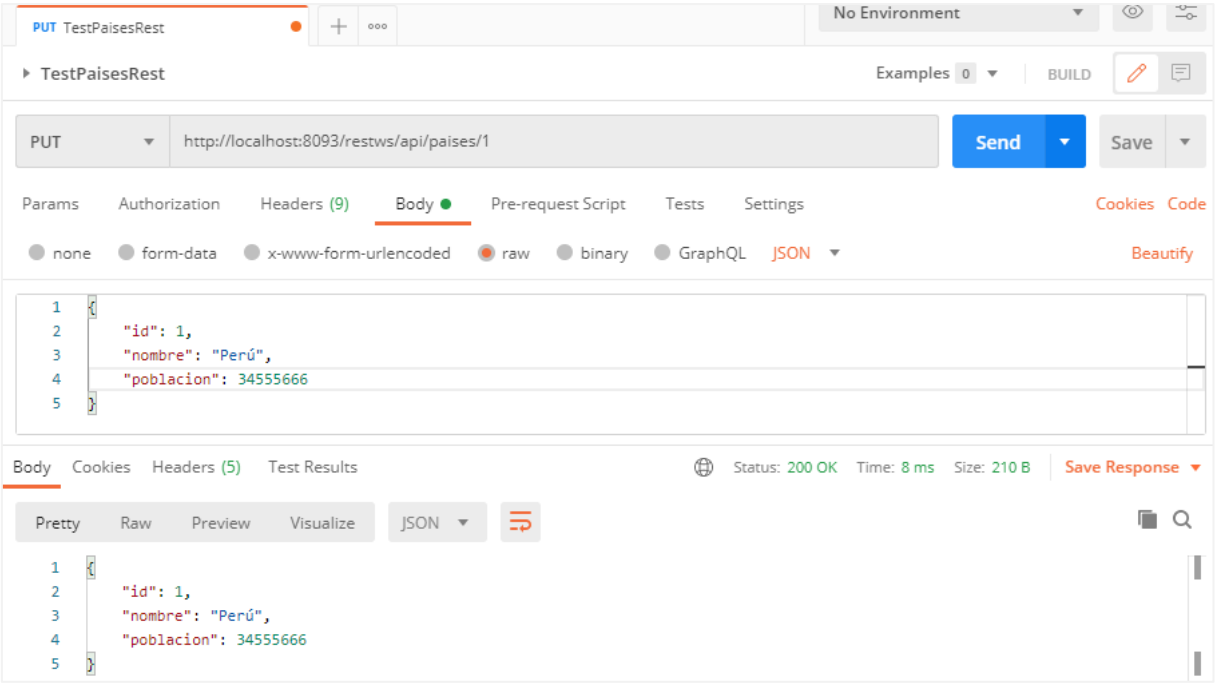


5. Para registrar un nuevo país, seleccione el tipo POST y en la sección *Body* seleccione la opción *Raw* para y el contenido en formato JSON. Luego presione él botón *Send*.

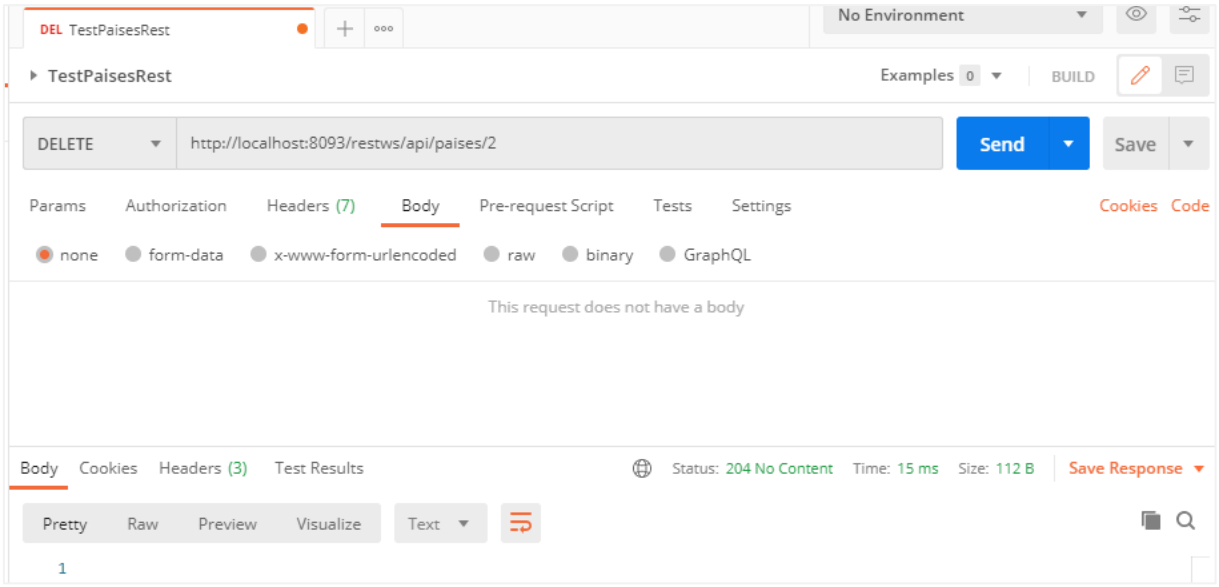


Verifique que se haya generado un nuevo *id*.

6. Para actualizar la información del país Perú, enviaremos el contenido en formato JSON incluyendo el id correspondiente y una población diferente.



7. Para eliminar un país, seleccionamos el método DELETE y agregamos el parámetro *id* sin contenido en el *body*.



Compruebe que el código devuelto es el 204, el cual fue colocado en el método *eliminar* del servicio RES

Actividad:

1. Cree una aplicación web que contenga un servicio RESTful que realice un CRUD básico a una entidad (registrar, actualizar, eliminar, listar y obtener por ID). La entidad debe tener como mínimo 5 campos y uno de ellos debe ser un identificador.

2. Cree una aplicación que consuma los métodos creados en la actividad N° 1.

3. En Postman, realice pruebas de 2 servicios REST con la información de Transantiago y analice la información devuelta.

   Documentación: https://gist.github.com/radutzan/a29aa8fb30b1b866bd0bc44d65a3676e